

Vulnerability Assessment Report

Imagur

Imagur (FShare)
132 Morphett Rd
Glengowrie SA 5044, Australia

Flinders University
Sturt Road, Bedford Park SA
5042

September 17, 2026

Report Version: 1.0

Table of Contents

List of Figures	3
1 Document Control	4
1.1 Team	4
1.2 List of Changes	4
2 Executive Summary	5
2.1 Overview	5
2.2 Identified Vulnerabilities	5
3 Methodology	7
3.1 Objective	7
3.2 Scope	7
3.3 User Accounts and Permissions	7
4 Findings	8
C1: Hardcoded AWS Root Credentials Exposed in Client-Side JavaScript	8
H1: Stored Cross-Site Scripting (XSS) via Uploaded Filename	9
H2: S3 Bucket Publicly Listable and Readable (Private Images Exposed)	10
M1: Account Enumeration via Registration	11
M2: Insecure Direct Object Reference via Hidden user_id Field in Upload	12
L1: Missing Cross-Site Request Forgery (CSRF) Protections	13
I1: Server and Technology Version Disclosure	14
5 Disclaimer	15
A Appendix	16
A.1 Static Appendix Section	16
A.2 Assessment Methodology and Limitations	16

List of Figures

Figure 1 - Distribution of identified vulnerabilities	6
---	---

1 Document Control

1.1 Team

Contact	Details	Role
ClawSec	Autonomous agent	Pentester

1.2 List of Changes

Version	Description	Date
1.0	Final report	Sep 17, 2026

2 Executive Summary

2.1 Overview

The "Imager" image-sharing service (<https://imagesharer.xyz/>, also listed as "FShare") is a public-facing web application that lets users upload and share images. An authorized security assessment was performed on 17 September 2026 to evaluate the security of the application and its supporting cloud infrastructure.

The assessment identified seven issues, including three critical/high-severity findings that require immediate attention:

1. **Critical — Hardcoded AWS root credentials in client-side code.** The website publicly serves a JavaScript file containing valid AWS access credentials that resolve to the AWS account's ROOT identity. Anyone who visits the site can obtain these keys and gain full administrative control of the business's AWS account, including all stored data, services, and billing.
2. **High — Private images are publicly downloadable.** The Amazon S3 bucket that stores all user images ("filesharer-research") allows anonymous listing and download. This means images that users marked "Private" can be viewed by anyone, bypassing the application's privacy controls.
3. **High — Stored cross-site scripting (XSS).** An attacker can upload an image whose filename contains a script that runs in the browser of anyone who opens the image link, potentially taking over their account.

In addition, the assessment found medium/low-severity issues including an ownership-check flaw in the upload form, missing CSRF protections, account enumeration during registration, and disclosure of server software versions.

Highest-priority actions: rotate and remove the exposed AWS root keys immediately, restrict the S3 bucket so it is not publicly listable or readable, and fix the filename output encoding that enables the XSS.

A technical breakdown of each finding, with remediation steps, is provided later in this report.

2.2 Identified Vulnerabilities

#	CVSS	Description	Page
C1	10.0	Hardcoded AWS Root Credentials Exposed in Client-Side JavaScript	8
H1	8.2	Stored Cross-Site Scripting (XSS) via Uploaded Filename	9
H2	7.5	S3 Bucket Publicly Listable and Readable (Private Images Exposed)	10
M1	5.3	Account Enumeration via Registration	11
M2	4.3	Insecure Direct Object Reference via Hidden user_id Field in Upload	12
L1	3.1	Missing Cross-Site Request Forgery (CSRF) Protections	13
I1	0.0	Server and Technology Version Disclosure	14

Vulnerability Overview

In the course of this penetration test **1 Critical**, **2 High**, **2 Medium**, **1 Low** and **1 Info** vulnerabilities were identified:

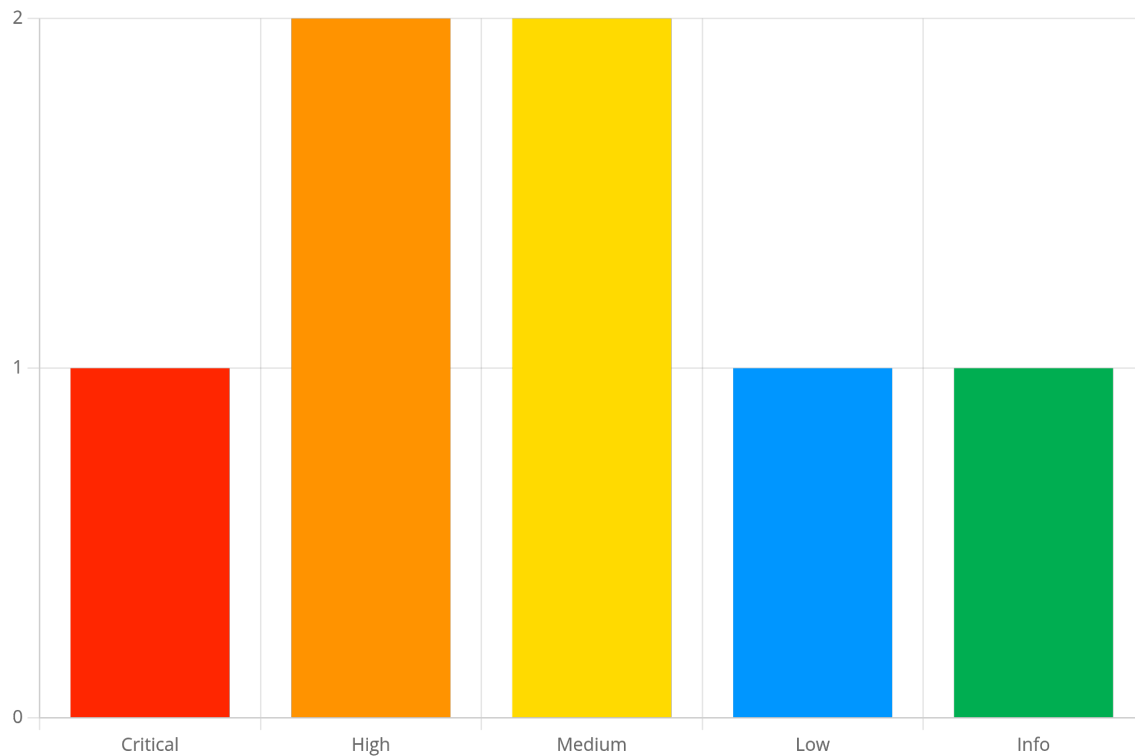


Figure 1 - Distribution of identified vulnerabilities

3 Methodology

This entire vulnerability assessment process was completed using agentic AI and existing LLM models. The purpose of such assessments is to automatically

3.1 Objective

The entire objective of this research is to evaluate the effectiveness of autonomously evaluate security posture for any business available on Google Map without the needs of technical guidance to determine how feasible, seamless and cost-affective such approach is.

3.2 Scope

The assessment covered the following in-scope systems and assets:

System	Description
https://imagesharer.xyz/	Primary web application ("Imager" image-sharing service), nginx/PHP on ports 80/443
https://www.imagesharer.xyz/	<code>www</code> subdomain, same nginx/PHP stack
http://51.161.131.44:8080/	Secondary instance, Apache/PHP on port 8080
resources.imagesharer.xyz	Amazon CloudFront distribution (d273uevi0662l0.cloudfront.net)
<code>filesharer-research</code>	Amazon S3 bucket (ap-southeast-2) storing user-uploaded images
<code>/js/awsvalid.js</code>	Client-side JavaScript credential file served by the web application

3.3 User Accounts and Permissions

Provided Users

- Unauthenticated (public) access
- One self-registered test account used for authenticated testing
- No production credentials were provided or required.

4 Findings

C1: Hardcoded AWS Root Credentials Exposed in Client-Side JavaScript	
Score	10.0 (Critical)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H
Target	https://imagesharer.xyz/js/awsvalid.js (AWS account 016170038077, region ap-southeast-2)
References	• https://imagesharer.xyz/js/awsvalid.js • AWS STS GetCallerIdentity

Overview

The public-facing website ships a JavaScript file containing hardcoded AWS access credentials. The credentials are valid and map to the AWS account root identity, giving any visitor full administrative control over the business's AWS account.

Details

The file `/js/awsvalid.js` (publicly downloadable at <https://imagesharer.xyz/js/awsvalid.js>) contains a hardcoded AWS Access Key ID, Secret Access Key, and region (`ap-southeast-2`) in plaintext comments. Validation via a read-only `aws sts get-caller-identity` call confirmed the credentials are active and resolve to `arn:aws:iam::016170038077:root` (account 016170038077, user id 016170038077) — the AWS account root identity. Anyone who fetches the JavaScript file can authenticate to the AWS account as root and perform unrestricted actions (read/modify/delete all resources, access stored data, create IAM users, incur costs). Only the minimum read-only proof (identity resolution) was obtained; no further AWS enumeration or modification was performed.

Recommendation

1. Immediately rotate and remove the exposed AWS root access keys.
2. Never embed AWS credentials in client-side code; issue temporary, least-privilege credentials server-side (STS / Cognito / IAM Roles).
3. Audit AWS CloudTrail for unauthorized activity and any IAM principals/resources created with the exposed keys.
4. Remove credential-bearing files from the public web root and scan the codebase (including git history) for hardcoded secrets.

H1: Stored Cross-Site Scripting (XSS) via Uploaded Filename

Score	8.2 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:L/A:N
Target	https://imagesharer.xyz/images/<uuid> (image detail page)
References	• OWASP A03:2021 Injection • CWE-79

Overview

The filename of an uploaded image is rendered without HTML escaping on the image detail page, allowing an attacker to store a JavaScript payload that executes in the browser of any user who opens the image link.

Details

Uploading an image with a filename such as `<img src=x onerror=alert(document.domain)>.png` results in the filename being reflected unescaped on the image detail page inside a `<span>` element, creating a live `<img src=x onerror=...>` element. The payload was verified to execute in a browser (an alert fired showing the domain). The dashboard view escapes the filename, but the detail page does not. Because the application has no CSRF protection and the XSS executes within an authenticated session, an attacker can perform actions on behalf of the victim (delete images, change visibility, read account data). Session cookies are flagged HttpOnly, which limits direct cookie theft but does not prevent same-origin account actions.

Recommendation

1. HTML-encode all user-supplied values (including filenames) on output using a context-aware templating engine (e.g. `htmlspecialchars/Blade` escaping).
2. Store the display filename separately from the on-disk name, and sanitize/reject control characters in filenames on upload.
3. Serve uploaded images from a separate origin with a restrictive Content-Type and Content-Disposition.

H2: S3 Bucket Publicly Listable and Readable (Private Images Exposed)

Score	7.5 (High)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Target	Amazon S3 bucket "filesharer-research" (ap-southeast-2)
References	https://filesharer-research.s3.ap-southeast-2.amazonaws.com/

Overview

The S3 bucket that stores all user-uploaded images allows anonymous listing and anonymous download, so every image — including those marked "Private" — is accessible to anyone without authentication, bypassing the application's privacy controls.

Details

The application stores uploaded images in the S3 bucket `filesharer-research` under the path `users/<user-uuid>/<image-uuid>.<ext>`. Two misconfigurations were confirmed: (1) an unauthenticated `ListObjectsV2` request (`?list-type=2`) returns the full object namespace (HTTP 200), exposing all object keys, user UUIDs, filenames, sizes, and timestamps; (2) an unauthenticated `GET` of an object returns the object contents (HTTP 200) without any pre-signed URL or signature. This was proven by anonymously downloading a "Private" test image. The application enforces privacy only at the web-application layer (the detail page redirects unauthenticated/foreign users to `/dashboard`), so the storage-layer exposure completely bypasses the privacy model. Multiple real user accounts (distinct user UUID prefixes) were present in the listing, confirming real user data exposure. Anonymous write (PUT) is correctly denied (403 AccessDenied).

Recommendation

1. Remove the public-read bucket policy; block public access (ListBucket and GetObject) at the S3 bucket level.
2. Serve images only through the application with per-object authorization, or via short-lived pre-signed URLs.
3. If public images are required, separate them into a dedicated public bucket and never store private objects there.
4. Enable S3 server access logging and CloudTrail to detect unauthorized access.

M1: Account Enumeration via Registration

Score	5.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Target	https://imagesharer.xyz/register
References	• CWE-204 • OWASP A01:2021

Overview

The registration form reveals whether an email address is already registered, allowing an unauthenticated attacker to enumerate valid accounts.

Details

Submitting a registration with an already-registered email returns the distinct message "An account with that email already exists." instead of a generic error. This allows attackers to confirm which email addresses have accounts on the platform, facilitating targeted attacks and privacy leakage.

Recommendation

1. Return a generic error (e.g. "Unable to complete registration") and send a confirmation email when the address is already in use, without revealing account existence.
2. Apply rate limiting to the registration endpoint.

M2: Insecure Direct Object Reference via Hidden user_id Field in Upload

Score	4.3 (Medium)
Vector string	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:L/A:N
Target	https://imagesharer.xyz/images/upload
References	• OWASP A01:2021 Broken Access Control • CWE-639

Overview

The image upload form trusts a hidden, attacker-controlled `user_id` field instead of deriving ownership from the authenticated session, allowing any logged-in user to upload images that are attributed to another user's account.

Details

The upload form includes `<input type="hidden" name="user_id" value="...">`. Testing showed that when a request was submitted with one account's session cookie but a different account's `user_id`, the uploaded image was stored under the other user's S3 prefix and appeared in the other user's gallery (not the requester's). This confirms the server trusts the client-supplied `user_id` for ownership attribution. An attacker can inject arbitrary images into any victim's gallery (including abusive or policy-violating content attributed to the victim). Combined with the publicly listable S3 bucket, victim user UUIDs can be enumerated to select targets.

Recommendation

1. Ignore any client-supplied `user_id`; derive the owner from the authenticated session only.
2. Enforce server-side authorization checks on every object-level operation (ownership must match the session user).

L1: Missing Cross-Site Request Forgery (CSRF) Protections

Score	3.1 (Low)
Vector string	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N
Target	https://imagesharer.xyz/ (logout, images/upload, images/<id>/visibility, images/<id>/delete, profile/avatar)
References	• OWASP A05:2021 Security Misconfiguration • CWE-352

Overview

State-changing forms and endpoints do not include CSRF tokens and do not validate the Origin/Referer header, allowing an attacker to craft requests that cause a logged-in user's browser to perform unintended actions.

Details

Review of the logout, image upload, visibility-change, delete, and avatar endpoints found no CSRF token on any form, and requests without an Origin/Referer header were accepted. An attacker can therefore trick an authenticated user into performing actions such as deleting their own images or changing an image's visibility. Object-level ownership checks do mitigate cross-account impact, so the practical impact is limited to actions on the victim's own account (denial of service, privacy toggle abuse).

Recommendation

1. Add per-session CSRF tokens to all state-changing forms/endpoints and validate them server-side.
2. Enforce SameSite=Strict (or Lax plus Origin/Referer validation) on session cookies.
3. Verify the Origin/Referer header for sensitive actions.

I1: Server and Technology Version Disclosure

Score	0.0 (Info)
Vector string	N/A
Target	https://imagesharer.xyz/ and http://51.161.131.44:8080/
References	CWE-200

Overview

The web servers disclose their software versions in error responses and response headers.

Details

Error responses disclose `Apache/2.4.68 (Debian)` and the `X-Powered-By: PHP/8.2.33` header identifies the PHP version, on both the nginx (80/443) and Apache (8080) instances. The Apache 8080 instance directly exposes the Apache version. This information assists attackers in selecting version-specific exploits.

Recommendation

1. Disable server version disclosure (ServerTokens Prod / ServerSignature Off for Apache; `server_tokens off` for nginx).
2. Remove or suppress the `X-Powered-By` header (`expose_php = Off`).

5 Disclaimer

The purpose of this research project is to help evaluating the seamlessness, effectiveness and cost-benefit ratio of autonomously performing vulnerability assessment using agentic AI. I do not take any responsibility upon any misuse or abuse of such tool where it can face significant legal actions.

A Appendix

A.1 Static Appendix Section

A.2 Assessment Methodology and Limitations

This assessment was conducted over a single engagement session on 17 September 2026 using a combination of reconnaissance and OSINT, DNS/WHOIS lookups, service fingerprinting, manual and scripted web application testing (SQL injection, cross-site scripting, broken access control / IDOR, cross-site request forgery, file upload handling, and Amazon S3 permission review), cloud storage review, and minimal read-only proof-of-concept validation.

The following limitations applied:

- Single engagement window (no continuous monitoring or retesting).
- No destructive testing was performed.
- No brute force, credential stuffing, or password spraying.
- No lateral movement within the environment.
- No AWS resource enumeration beyond a single read-only identity check.
- No email (MX) infrastructure was present to assess.